Map Basics & Declaration

In Go, a map is a built-in data structure that associates (maps) keys of one type
to values of another type. Internally, Go implements maps as hash tables.

The map type is written as map[keyType]valueType.

Declaration Styles:

Nil Map:

```
var nilMap map[string]int
```

- Creates a nil map. Its length is 0.
- Reading from a nil map returns the value type's zero value.
- Danger: Attempting to write to a nil map causes a runtime panic.

Empty Map Literal:

```
totalWins := map[string]int{}
```

Creates a usable map with a length of 0 (not nil), which you can safely write to.

Populated Map Literal:

```
teams := map[string][]string {
    "Orcas": []string{"Fred", "Ralph", "Bijou"},
    "Lions": []string{"Sarah", "Peter", "Billa"},
    "Kittens": []string{"Waldo", "Raul", "Ze"},
}
```

Using make:

```
ages := make(map[int][]string, 10)
```

Creates a map with a default starting size (in this case, space for 10 elements),
but its length remains 0 until populated.

Key Constraints:

- A map key must be a comparable type.

- You cannot use slices, maps, or functions as map keys, as they are not comparable.

Reading, Writing, Deleting, and Emptying

Reading and Writing:

Done using bracket syntax (e.g., m["key"] = 10).

Missing Keys:

If you read a key that doesn't exist, Go does not throw an error;
it simply returns the zero value for the map's value type.
For example, totalWins["Kittens"]++ works perfectly even if "Kittens" isn't in the map yet,
because it resolves to 0++.

Deleting:

Use the built-in delete(m, "key") function. If the key isn't present or the map is nil, nothing happens.

Emptying:

Use the built-in clear(m) function to empty the map. Unlike slices,
calling clear() on a map actually sets its length to 0.

The Comma Ok Idiom

Because reading a missing key returns a zero value, you cannot simply check if m["world"] == 0
to know if "world" exists in the map (as its actual value might genuinely be 0).

To solve this, Go provides the comma ok idiom:

```
v, ok := m["hello"]
```

If ok is true, the key exists in the map and v is its value.
If ok is false, the key is not present, and v is the zero value.

Comparing Maps

Maps are not comparable with == or != (you can only compare them to nil).
To check if two maps have identical keys and values,
you should use the maps.Equal or maps.EqualFunc functions introduced in Go 1.21.

Using Maps as Sets

Go does not have a built-in "Set" data structure, but you can simulate one using a map where
the keys represent the set elements.

- Using bool: map[int]bool{}. You can check for membership by reading the key: if intSet { ... }.

- Using struct{}: map[int]struct{}{}. An empty struct consumes zero bytes of memory
(whereas a boolean uses 1 byte). However, using struct{} makes assignment
and checking slightly clumsier, requiring the comma ok idiom: if _, ok := intSet; ok { ... }.

Choose based on your memory vs. readability needs.

Pitfalls & Fixes

1: Inefficient Map Initialization

The Concept:

Under the hood, a Go map is an array of buckets, where each bucket holds up to 8 key-value pairs.
When a map gets too full (its "load factor" goes above 6.5) or has too many overflow buckets, it
automatically grows by doubling its number of buckets.

The Mistake:

When a map grows, all the existing keys must be redistributed (rehalanced) across the new buckets.
This is a heavy, O(n) computation. If you add 1 million elements to an empty map m := map[string]int{},
the map will have to stop and rebalance itself dozens of times.

The Fix:

If you know the approximate number of elements the map will hold up front, always initialize it with a size using make:

```
m := make(map[string]int, 1_000_000)
```

Providing a size hint asks the Go runtime to pre-allocate enough buckets to handle that many elements,
completely avoiding costly runtime reallocations and speeding up insertions by roughly 60%.

2: Maps and Memory Leaks

The Concept:

A Go map can grow and add buckets, but it never shrinks.

The Mistake:

If you add a massive number of elements to a map (causing it to allocate hundreds
of thousands of buckets) and then use delete() to remove all those elements,
the garbage collector will clean up the individual elements, but the buckets themselves remain in memory.

If you build a cache that experiences a massive temporary spike in data,
your application's baseline memory consumption will permanently increase.

The Fix:

1. Re-create the Map: If you expect temporary spikes, periodically build a brand-new map,
copy the retained elements over, and let the old map be garbage collected.

2. Use Pointers: If you must keep a massive number of buckets, change your value type to a pointer
(e.g., map[int]*byte instead of map[int]byte). This significantly reduces the memory
reserved for each bucket slot.

3: Wrong Assumptions During Map Iteration

The Mistake:

Developers coming from other languages often wrongly assume how for k, v := range m behaves.

1. Ordering:

Go intentionally scrambles the iteration order of maps using a random number generated when the map is created.
You cannot rely on keys being sorted, nor can you rely on them preserving their insertion order.
Furthermore, the order will change from one loop to the next.

2. Inserting during iteration:

If you insert a new key-value pair into a map while you are iterating over it,
the Go specification states that the new entry "may be produced during the iteration or skipped".
The behavior is totally non-deterministic.

The Fix:

1. If you need a predictable order, extract the keys into a slice, sort the slice,
and then iterate over the sorted keys to access the map values.

2. If you need to update a map while looping over it (and want to guarantee the
new elements aren't accidentally processed in the same loop), create a copy of the map,
iterate over the original, and write your updates to the copy.